

How does a program decide what to do next?

Warm Up

Objectives

By the end of this we should have a base understanding of flow control.



Conditional Statements

Learn to guide program execution using logic and decision-making structures.



For and While Loops

Master the ability to repeat tasks efficiently through iterative code blocks.

Instruction: Conditional Statements (if, elif, else)

What Are Conditional Statements?

Conditional statements allow a program to make decisions based on certain conditions. In Python, we use **if**, **elif**, and **else** to execute code blocks depending on whether a condition is **True** or **False**.

Why Are They Important?



Respond to Input

Checking if a password is correct to grant access.



Game Logic

Determining if a player wins based on their score.



Filter & Analyze

Selecting specific data points, like numbers > 10.

Basic Structure of an **If** Statement

1. The Basic **if**

The **if** statement checks a condition. If **True**, the block runs.

```
age = 18
if age >= 18:
    print("You are allowed to vote.")
```

Explanation: Since age is 18 (≥ 18), the message is printed.

2. Adding **else**

If the condition is **False**, the code in the **else** block runs instead.

```
age = 16
if age >= 18:
    print("You are allowed to vote.")
else:
    print("You are too young to vote.")
```

Explanation: Since 16 is < 18 , the **else** block executes.

Using `Elif`: Checking Multiple Conditions

Sometimes, we need to check multiple conditions in sequence. We use `elif` (short for "else if") to provide alternative branches.

```
grade = 85
if grade >= 90:
    print("You got an A!")
elif grade >= 80:
    print("You got a B.")
elif grade >= 70:
    print("You got a C.")
else:
    print("You need to improve.")
```

Logic Flow

Since `grade` is 85:

- 1. `grade >= 90` is **False**
- 2. `grade >= 80` is **True**
- 3. The `elif` block executes
- 4. Other branches are skipped

Output: "You got a B."

Complex Conditions: Comparison & Logical Operators

Combine conditions using comparison operators (<, >, ==, !=, >=, <=) and logical operators (**and**, **or**, **not**).

```
temperature = 75
is_raining = False

if temperature > 70 and not is_raining:
    # Condition is True
    print("It's a nice day to go outside!")
else:
    print("Better stay inside.")
```

How it Works

- **temperature > 70**: 75 is greater than 70, so this part is **True**.
- **not is_raining**: Since raining is False, "not False" becomes **True**.
- The **and** operator requires both sides to be **True**.

Result: "It's a nice day..."

Python Syntax: The Importance of Indentation

Python uses **indentation** (spaces or tabs) to define blocks of code inside control structures. Incorrect indentation results in an **IndentationError**.

Incorrect Example

```
if x > 10:  
print("x is greater than 10")
```

Missing indentation after the colon!

Correct Example

```
if x > 10:  
    print("x is greater than 10")
```

Indented block clearly defined.

Summary: Key Takeaways

 **If** statements run code only when a condition is **True**.

 **Else** statements run when the `if` condition is **False**.

 **Elif** statements check multiple conditions in order.

 **Comparison and logical operators** allow for complex conditions.

 **Proper indentation** is mandatory for code blocks.

Code Challenge: Predict the Output

Analyze the following Python snippet. What will be the final result printed to the console?

```
score = 92
if score >= 90:
    print("Excellent job!")
elif score >= 80:
    print("Great work!")
else:
    print("Keep practicing!")
```

What will be printed?



Select the correct outcome based on the variable `score = 92`.

Activity: Writing Conditional Statements

Objective

In this activity, you will practice writing conditional statements using `if`, `elif`, and `else` in Python. By the end of the activity, you should be able to:

- ✓ Write and execute basic `if-else` statements.
- ✓ Use `elif` to handle multiple conditions.
- ✓ Apply logical and comparison operators in conditions.

Part 1: Simple If-Else Statements

Task: Write a program that asks the user for their age and prints whether they are a minor (under 18) or an adult (18 and older).

Steps

- ✓ Use the `input()` function to ask for age.
- ✓ Convert input to integer using `int()`.
- ✓ Use `if-else` to check and print.

Example Output

```
Enter your age: 16  
You are a minor.
```

```
Enter your age: 21  
You are an adult.
```

REMEMBER! Try these activities first then check the code. Practice is key to learning.

Part 1: Solution Code

Task Recap: Checking if a user is a minor or an adult based on age input.

Python Solution

```
age = int(input("Enter your age: "))
```

```
if age >= 18:  
    print("You are an adult.")  
else:  
    print("You are a minor.")
```

Part 2: Using Elif for Multiple Conditions

Task: Write a program that checks a student's grade and assigns a letter grade based on a scale.

Grading Scale & Steps

90+: A | 80-89: B | 70-79: C

Below 70: Needs Improvement

- ✓ Ask user for grade (integer).
- ✓ Use **if-elif-else** statement.
- ✓ Print the letter grade result.

Example Output

```
Enter your grade: 85  
You got a B.
```

```
Enter your grade: 65  
You need to improve.
```

Part 2: Solution Code

Task Recap: Grading logic using if-elif-else statements in Python.

```
# Python Solution

grade = int(input("Enter your grade: "))

if grade >= 90:
    print("You got an A!")
elif grade >= 80:
    print("You got a B.")
elif grade >= 70:
    print("You got a C.")
else:
    print("You need to improve.")
```

Part 3: Using Logical Operators in Conditions

Task: Write a program that checks if a number is within the range of 1 to 10 (inclusive).

Steps

- ✓ Ask the user for a number.
- ✓ Use **if** and **and** to check the range.
- ✓ Print the result to the user.

Conditions

```
# If number in range [1, 10]
"The number is in range."

# Otherwise
"The number is out of range."
```

Part 3: Solution Code

Task Recap: Checking if a number is within a specific range using logical operators.

```
# Python Solution

number = int(input("Enter a number: "))

if number >= 1 and number <= 10:
    print("The number is in range.")
else:
    print("The number is out of range.")
```


Wrap-up: Reflection & Submission

Reflection Questions



What happens if you forget to use `elif` and just use multiple `if` statements?



How does Python decide which block of code to execute in an `if-elif-else` structure?



Why is indentation important when writing conditional statements in Python?



Save and submit your completed notebook.

Hands-on Activity: Nested Conditionals

Task: Modify the grade-checking program to check for a perfect score (100) and print **"You got a perfect score!"**



Hint

Use a **nested if** statement or an additional condition (logical operator) to check for a perfect score.

Consider where this check should live in your **if-elif-else** chain.



**Save and submit your completed notebook.
Ensure it runs from start to finish.**

Instruction: Loops (For and While)

What are Loops? Loops allow us to repeat a block of code multiple times without having to write it repeatedly.

For Loops

Used when we know **exactly how many times** we want to repeat an action.

```
# Example
for i in range(5):
    print("Hello")
```

While Loops

Used when we want to repeat an action **until a certain condition** is met.

```
# Example
while count < 5:
    print("Hello")
    count += 1
```

Instruction: For Loops

What is it? A for loop is useful when we need to iterate over a sequence of items (like a list, string, or range of numbers).

Basic Syntax

The standard structure in Python:

```
for variable in sequence:  
    # Code to repeat
```

How it Works

- The loop runs once for each item in the sequence.
- The variable takes on the value of each item one by one.

Instruction: For Loop Examples

Example 1: Looping Through a List

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output:

- apple
- banana
- cherry

Example 2: Using a Range

We can use the range() function to loop a specific number of times.

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Instruction: While Loops

What is it? A while loop runs as long as a certain condition remains True. The loop continues until the condition becomes False.

Example 1: Counting Up to 5

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output:

1, 2, 3, 4, 5

Example 2: User Input Loop

```
user_input = ""
while user_input.lower() != "stop":
    user_input = input("Type something...")
```

Functionality:

Repeats until "stop" is entered.

When to Use For vs. While Loops

Use Case	For Loop	While Loop
You know how many times to repeat the code	✓ Yes	✗ No
You need to iterate over a sequence (list, range, string, etc.)	✓ Yes	✗ No
You want to repeat code until a condition changes	✗ No	✓ Yes
The loop might run indefinitely if not careful	✗ No	✓ Yes

Instruction: Loop Control

Break and Continue in Loops

Python has two important loop control statements that alter the normal flow of execution:

break

Definition:

Stops the loop immediately.

Example:

```
for x in range(10):  
    if x == 5:  
        break  
    print(x)
```

continue

Definition:

Skips the current iteration and moves to the next one.

Example:

```
for x in range(5):  
    if x == 2:  
        continue  
    print(x)
```


Examples: Loop Control in Action

Using break to Stop Early

```
for number in range(10):  
    if number == 5:  
        break # Stops loop at 5  
    print(number)
```

Output:

```
0, 1, 2, 3, 4
```

Using continue to Skip

```
for number in range(5):  
    if number == 2:  
        continue # Skips printing 2  
    print(number)
```

Output:

```
0, 1, 3, 4
```

Debugging: Common Loop Mistakes

1. Infinite While Loops

- Forgot to update the loop condition.
- Loop runs forever because condition is always True.
- **Fix:** Ensure condition eventually becomes False.

```
count = 1
while count > 0: # Always True
    print(count)
```

2. Off-by-One Errors

`range(n)` stops **before** `n`.

To loop exactly `n` times, use `range(n)`, not `range(n+1)`.

3. Misusing break

Ensure `break` is placed inside an `if` statement.

Without a condition, the loop will exit immediately on the first iteration.

Conclusion: Summary & Next Steps

For Loops

Great for iterating over sequences or repeating code a set number of times.

While Loops

Best when looping until a specific condition is met.

Loop Control

Break stops a loop early, while **continue** skips an iteration.

What's Next?

Next, you will use these techniques to automate tasks and analyze data using Python

Real-World Applications

These problems may seem simple, but they reflect the same logic used in real-world systems like:



Recommendation Engines

Suggesting content based on user patterns.



Data Pipelines

Moving and transforming large datasets.



Decision Systems

Automating complex logic at scale.

Activity: Looping through Data

Objective

In this activity, you will practice using `for` and `while` loops to iterate through different types of data, such as:

- Lists, strings, and numerical ranges
- Analyzing and processing data sets
- Solving real-world problems with automation
- Loops are essential in data science because they allow us to process large amounts of data efficiently.
-

Part 1: Looping Through a List

Task 1: Print Items

A **for loop** is commonly used to iterate through lists.

- Create a list of five of your favorite foods.
- Use a for loop to print each item on a separate line.

Example Output:

Pizza, Sushi, Pasta, Burgers, Tacos

Task 2: Modify Items

Practice transforming data within a loop.

- Create a list of numbers from 1 to 5.
- Use a for loop to print each number doubled (e.g., 2, 4, 6, ...).

Part 2: Looping Through a String

Task 3: Count Vowels in a Sentence

A `for loop` can also be used to iterate through each letter in a string.

- Ask the user to enter a sentence.
- Use a for loop to count how many vowels (a, e, i, o, u) are in the sentence.
- Print the total number of vowels.

Example Execution

Input:

Enter a sentence: `I love learning Python!`

Output:

`Total vowels: 7`

Part 3: Using a While Loop

Task 4: Guess the Number Game

A **while loop** is useful when we don't know how many times a loop needs to run.

- Set a random number between 1 and 10.
- Ask the user to guess the number.
- Use a while loop to keep asking until they guess correctly.
- Print a success message when they guess correctly.

Example Execution

Guess a number between 1 and 10: 4

Too low! Try again.

Guess a number between 1 and 10: 7

Too high! Try again.

Guess a number between 1 and 10: 5

Correct!

Part 4: Using Break and Continue

Task 5: Stop When a Certain Word is Found

Sometimes we need to stop a loop early or skip an iteration.

- Create a list of words: `["apple", "banana", "cherry", "stop", "grape"]`
- Loop through the list and print each word.
- If the word "stop" is found, **break** out of the loop.

Expected Output:

```
apple  
banana  
cherry
```

(The loop stops before printing "stop" or "grape".)

Activity Summary

In this activity, you used **for loops** and **while loops** to:

- Iterate through lists and strings
- Use loops to process user input
- Implement **break** to stop a loop early

Submit your Jupyter notebook for credit.

Activity: Flow Control

Read Chapter 2 of *Automate the Boring Stuff*

In this reading, you'll look at the different ways that you can control which code activates in your code cell, at what time, and how many times it activates.

This is an important skill for any programmer to learn as we will see this throughout the coding journey.

[Access Chapter 2 Online](#)